PROJECT ARIA · DATA HUB (LAYER 2) · SLICE 1

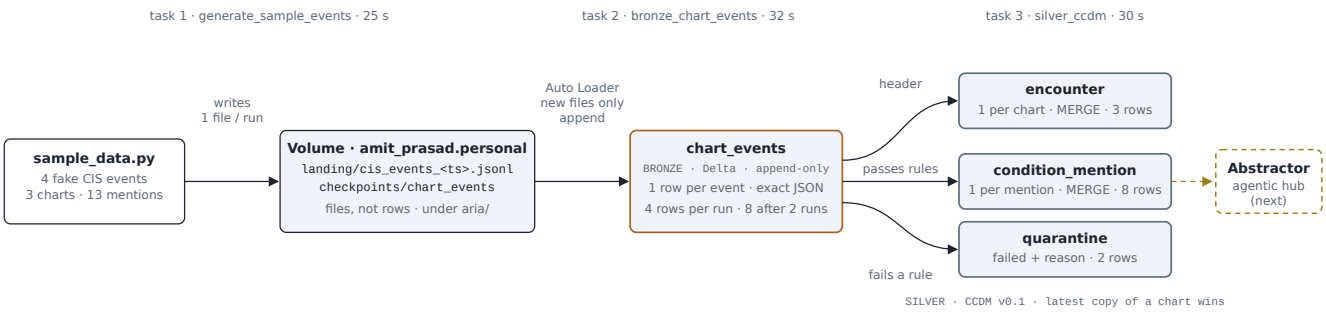
ARIA Slice 1 Demo

One Databricks job, three tasks, four tables. Synthetic CIS chart events land in a Volume, Auto Loader copies them untouched into bronze, and silver turns them into the first CCDM tables. Everything below is what actually ran on Sep 23, 2026 in ai-engineering-dev.

JOB	RUN	DURATION	COMPUTE
[dev amit_prasad] aria_hello_pipeline	201202913316971 · Sep 23, 13:57	1 m 29 s · **Succeeded**	Serverless · env version 5

CATALOG · SCHEMA	LINEAGE
ai_engineering_dev_catalog_1688890232235261 · amit_prasad	1 upstream · 4 downstream tables

The picture



One run, left to right. Task 1 exists only because CIS/Kafka is not connected yet; in prod it is switched off and the Kafka reader takes the place of the Volume. The rules that decide "passes" or "fails" live in `rules.py`: valid ICD-10 format, assertion in the CCDM vocabulary, page present, span start < end.

What happens when you press Run

Before the run there is a deploy: VS Code runs `databricks bundle deploy -t dev`, which uploads the project to `/Workspace/Users/amit.prasad@cotiviti.com/.bundle/aria_data_pipeline/dev/files` and creates or updates the job from `resources/aria_hello_pipeline.job.yml`. Nobody clicks a job together in the UI. Then, in order:

1. **The job starts on serverless** with its parameters (catalog, schemas, Volume path, CCDM version, `generate_samples`). Each notebook reads them as widgets and `config.py` turns them into paths and three-level table names. Nothing is hard-coded, so the same code runs in mvp and prod with different values.
   `databricks.yml variables → job parameters → dbutils.widgets → Settings`

2. **generate_sample_events** writes one JSON-lines file with 4 events into the Volume landing folder: CH-8841, CH-8842, CH-8843, and CH-8841 again as a re-send with one extra mention (E78.5). CH-8843 carries two deliberately bad mentions (code `XYZ`, assertion `maybe`). Runs only when `generate_samples = true`.
   `00_generate_sample_events.py → sample_data.write_sample_file()`

3. **bronze_chart_events** creates `chart_events` if it is missing, then Auto Loader (`cloudFiles`, text format, `availableNow`) reads the files it has not seen before and appends one row per line: the exact JSON in `payload`, plus chart_id, client_id, received_at and source_file pulled out for convenience. No de-duplication, no validation. Plus 4 rows every run.
   `10_bronze_chart_events.py → bronze.ingest_chart_events()`

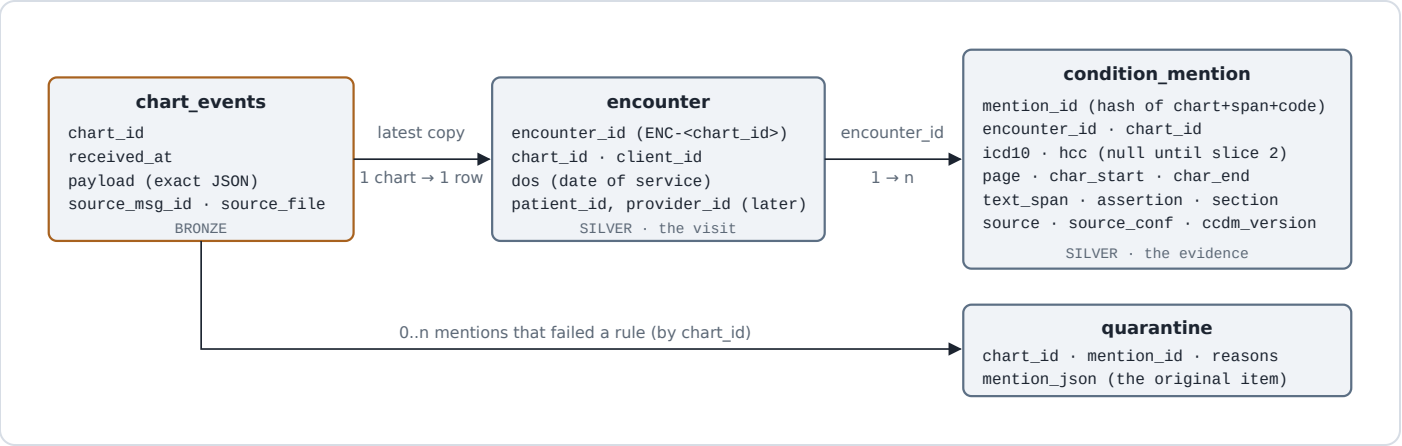
4. **silver_ccdm** creates the three CCDM tables if missing, reads bronze, keeps the latest copy of each chart (newest `received_at`), parses the JSON with a fixed schema, and splits it: the header becomes one `encounter` row (MERGE on chart_id); each item of `nlp_output` becomes one mention row, the assertion is translated (present → ACTIVE, history → HISTORY, negated → NEGATED, possible → SUSPECTED), the rules run, and the mention goes to `condition_mention` (MERGE on chart_id + mention_id) or to `quarantine` with its reasons. A chart's old rows are deleted before its new ones are loaded, so re-running never duplicates.
   `20_silver_ccdm.py → silver.run() → quality.assert_no_nulls / assert_min_rows`

5. **Quality gates** check the result (no nulls in chart_id, mention_id, icd10, assertion; at least one row). If they fail, the task fails and the email notification fires instead of bad data flowing on.

TABLE	LAYER	AFTER RUN 1	AFTER RUN 2	WHY
chart_events	bronze	4	8	append-only; every run lands a new file
encounter	silver	3	3	one per chart; the two copies of CH-8841 collapse to one
condition_mention	silver	8	8	10 mentions in the latest copies, 8 pass
quarantine	silver	2	2	CH-8843: `XYZ` (icd10_invalid), `maybe` (assertion_not_in_vocabulary)

The sentence to say with that table: bronze grows with every run; silver only changes when the charts change.

The tables and how they relate



Header table and detail table. A visit has one date and one client but many mentions, so the header lives once in `encounter` and each piece of evidence is its own row in `condition_mention`. Quarantine keeps the failed rows instead of dropping them.

Why they are separate tables

encounter is the visit: one patient, one provider, one date of service. Visit-level facts change together and will be enriched from other feeds (patient, provider, later "face-to-face" and "in the payment year"). Storing them on every mention row would repeat them fifty times and make a correction a fifty-row update.

condition_mention is the evidence: every place the chart mentions a condition, pinned to page and character span, with the assertion. "No evidence of DVT" and "DVT" carry the same code with opposite meaning; keeping one row per mention, with the assertion, is what lets a coder or an auditor check each one and lets the gold layer roll them up into a decision with provenance.

quarantine is the reject side: a mention that fails a rule goes here with the reason while the rest of its chart flows on. One bad code never blocks a whole visit, and nothing disappears silently.

The files

`src/aria/ · the library (tested on the laptop)`

`config.py`

`Settings`: job parameters in, paths and table names out (`landing_path`, `checkpoint_root`, `bronze_table()`, `ccdm_table()`).

`sample_data.py`

The four fake CIS events and `write_sample_file()`. Dev only.

`bronze.py`

Creates `chart_events`; Auto Loader read; append. The only file that knows where data comes from (swap for Kafka later).

`rules.py`

The business rules in one place: assertion translation map, vocabulary, ICD-10 format check, stable `mention_id`, quarantine reasons. Stand-in for the CCDM reference tables until slice 2.

`silver.py`

Creates the three CCDM tables; latest copy per chart; parse; build encounters and mentions; apply rules; delete-then-MERGE.

`quality.py`

`assert_no_nulls`, `assert_min_rows`: fail the task instead of passing bad data.

`src/notebooks/ · the three tasks (thin)`

`00_generate_sample_events.py`

Read widgets → write the sample file if `generate_samples` is true.

`10_bronze_chart_events.py`

Read widgets → `bronze.ingest_chart_events()` → show row counts.

`20_silver_ccdm.py`

Read widgets → `silver.run()` → quality gates → display the tables.

`resources/ and root · the deployment`

`resources/aria_hello_pipeline.job.yml`

The job: three tasks in order, serverless, parameters, retries, email on failure. Deploy turns this into the job in Jobs & Pipelines.

`databricks.yml`

The bundle: name, which resource files to include, variables with dev defaults, targets dev / mvp / prod (dev = your prefix and personal schema; prod = real schemas, service principal).

`tests/ · Jenkinsfile · CLAUDE.md`

pytest for rules and transforms; CI/CD placeholder to be replaced from the reference repo; the rules file Security asks for.

Demo script (about five minutes)

1. **VS Code, project tree.** Point at `databricks.yml`, `resources/`, `src/aria`, `src/notebooks`, `tests/`.

   > "*This is a Databricks Asset Bundle. The YAML describes the job, the Python package holds the logic, the notebooks are three thin entry points, and the tests run on my laptop without a cluster.*"

2. **Deploy and Run** from the Bundle Resource Explorer, or open the last run. Three green boxes, 1 m 29 s, serverless.

   > "*Deploy uploads the code and creates the job from the YAML; nothing is built by hand in the UI. The job ran on serverless, so there is no cluster to start or pay for between runs.*"

3. **Catalog Explorer → Volume** `amit_prasad.personal` → `aria/landing`: one file per run; `aria/checkpoints`: Auto Loader's memory.

   > "*Today a generator drops the file; in production CIS publishes to Kafka and only the reader in bronze.py changes.*"

4. **SQL Editor, the counts.** Run the query below. Then run the job again and run it once more: bronze goes up by 4, the three silver tables do not move.

   > "*Bronze is the replay record and keeps every copy. Silver is idempotent: the latest copy of a chart wins and MERGE keys are stable, so re-running never duplicates.*"

```
SELECT 'chart_events' t, count(*) n FROM ai_engineering_dev_catalog_168889023223526
UNION ALL SELECT 'encounter', count(*) FROM ai_engineering_dev_catalog_16888902322
UNION ALL SELECT 'condition_mention', count(*) FROM ai_engineering_dev_catalog_1688
UNION ALL SELECT 'quarantine', count(*) FROM ai_engineering_dev_catalog_16888902322
```

5. **The mentions.** Show CH-8842's three rows and point at "no evidence of DVT" → NEGATED. Then the quarantine rows with their reasons.

   > "*Same code, opposite meaning. The assertion travels with every mention, and the page and character span let a reviewer open the PDF at the sentence. The two bad mentions are kept with a reason instead of being dropped.*"

```
SELECT chart_id, page, char_start, icd10, assertion, text_span
FROM ai_engineering_dev_catalog_1688890232235261.amit_prasad.condition_mention
ORDER BY chart_id, page, char_start;

SELECT chart_id, reasons, mention_json
FROM ai_engineering_dev_catalog_1688890232235261.amit_prasad.quarantine;
```

6. **Lineage tab** on `condition_mention`: `chart_events` → the job → the table. Unity Catalog recorded it without any extra code.

7 **Close.**

> "*condition_mention is the hand-off to the Abstractor. Next: the CCDM reference tables (vocabulary, source_map, code_reference) so the mapping is data instead of code, then provider and evidence_span, then the Kafka reader in place of the generator, and the repo goes into Bitbucket with Jenkins deploying to dev, mvp and prod.*"

Questions you will get

Why keep the duplicate copy of CH-8841 in bronze?

Bronze is what we received, not what we believe. If a rule or the CCDM changes, silver is rebuilt from bronze; if CIS disputes what it sent, the exact payload is there. The de-duplication happens once, in silver, by newest received_at.

Why Auto Loader instead of a plain read of the folder?

Auto Loader remembers which files it has loaded (the checkpoint), so each file is loaded exactly once and a run with nothing new adds nothing. A plain read would re-load every file every run.

Where are the reference tables?

Not tables yet. The assertion map, vocabulary and ICD-10 check are constants in `rules.py` with a comment naming the table each one stands in for. Slice 2 makes them Delta tables (`ccdm.vocabulary`, `ccdm.source_map`, `ccdm.code_reference`) seeded from CSV, and silver joins to them. A new NLP vendor then means new rows, not new code.

Is condition_mention the final output?

It is the final output of silver and the input to the Abstractor. The data hub also hosts what comes after: the CTG store, the Auditor's gates, gold `determination` and the egress back to CIS. None of that exists yet.

Why is the job called [dev amit_prasad] …?

Bundle development mode: every developer gets a prefixed copy of the job, tables in their own schema, and schedules paused. Jenkins deploys the unprefixed job in production mode as the service principal.

What broke?

The first run failed in silver with `PERSIST TABLE is not supported on serverless compute`: a `.cache()` on the parsed events. Serverless does not allow cache or persist; removing it fixed it. Worth remembering for every job we put on serverless.

Where is the PHI?

Nowhere. All events are invented (fake chart ids, textbook phrases). The real feed will carry PHI, which is why bronze already has `phi_flag` and the prod target uses governed schemas.

What comes next

1. **Version control and CI/CD.** Git for Windows, Bitbucket HTTP token, sandbox repo, push; adapt the Jenkinsfile from the reference repo; Jenkins onboarding and webhook so a merge to dev deploys to dev.
2. **Slice 2 of the code.** The three reference tables as Delta tables seeded from CSV, silver joining to them, `hcc` filled from `code_reference`; then `provider` and `evidence_span`.
3. **Ingress.** Kafka reader (`aria.chart.ready`) in place of the generator, once CIS can publish.

> Numbers above are from the sample file in `sample_data.py`. If the file changes, the expected counts change with it; the tests in `tests/` hold the same expectations.